

MACHINE LEARNING

Computational Physics Laboratory 2 – CPL 203

Data Set : A CERN Electron collision dataset
Software : Python (Google Colab)
Machine Learning techniques: Random Forest Regression, DNN

M.M.M.D. Ariyaratna

Ph.D. Student, CERN

Table of Contents

Introduction.....	2
Dataset.....	2
Methods.....	4
Preprocessing	4
Exploratory Data Analysis	6
Random Forest Regression	6
Deep Neural Network	6
Principle Component Analysis.....	7
Results and Discussion	7
Exploratory Data Analysis	7
Random Forest Regression	8
Deep Neural Network	9
Principle Component Analysis.....	9
Conclusion	11

Introduction

Particle collisions are produced when subatomic particles (such as leptons, protons, and ions) accelerate to very high energies and are made to collide with one another in particle accelerators (e.g., the LHC at CERN and RHIC at Brookhaven).

Particle collisions are crucial because,

- Study particles and forces at subatomic level
- Create new particles
- Test fundamental theories

Here, a few terms that are very important in high-energy physics (HEP) are introduced.

- **Momentum**

Momentum is a vector quantity that describes the motion of a particle.

$$\vec{p} = m\vec{v}$$

In relativistic physics,

$$E^2 = p^2c^2 + m^2c^4$$

This equation is important in high energy collisions. In collisions momentum is conserved. It is used to reconstruct particle trajectories and interactions.

- **Energy**

Energy gives a measurement of a particles ability to do works or create particles. It determines which particle can be created.

- **Charge**

Charge is a fundamental property of a particle that determines how a particle interact electromagnetically. It determines particle bending in EM fields which is the way to identify particle types. Charge is also conserved in particle collisions.

Modern HEP experiments generate very large datasets which are manually impossible to analyze. Machine learning helps to analyze those very large datasets. Machine learning can identify patterns in datasets, event classification and regression tasks.

Dataset

In this project, a CERN electron collision dataset was analyzed. The dataset contains 100,000 events from electron–positron collisions (the most common and physically significant collision type, which produces bosons), as well as electron–electron and positron–positron collisions.

Each data entry is characterized by the following 19 attributes.

1. Run : The run number of the event
2. Event : The event number

3. E1 : The total energy of the electron 1 (GeV)
4. px1 : The x component of the momentum of the electron 1 (GeV)
5. py1 : The y component of the momentum of the electron 1 (GeV)
6. pz1 : The z component of the momentum of the electron 1 (GeV)
7. pt1 : The transverse momentum of the electron 1 (GeV)
8. eta1 : The pseudo rapidity of the electron 1
9. phi1 : The phi angle of the electron 1
10. Q1 : The charge of the electron 1
11. E2 : The total energy of the electron 2 (GeV)
12. px2 : The x component of the momentum of the electron 2 (GeV)
13. py2 : The y component of the momentum of the electron 2 (GeV)
14. pz2 : The z component of the momentum of the electron 2 (GeV)
15. pt2 : The transverse momentum of the electron 2 (GeV)
16. eta2 : The pseudo rapidity of the electron 2
17. phi2 : The phi angle of the electron 2
18. Q2 : The charge of the electron 2
19. M : The invariant mass of two electrons (GeV) → Target

The invariant mass is the target variable. It can be calculated from energy and momentum of the two particles.

$$M^2 = (E_1 + E_2)^2 - (\vec{p}_1 + \vec{p}_2)^2$$

This is the distribution of invariant mass after preprocessing the dataset.

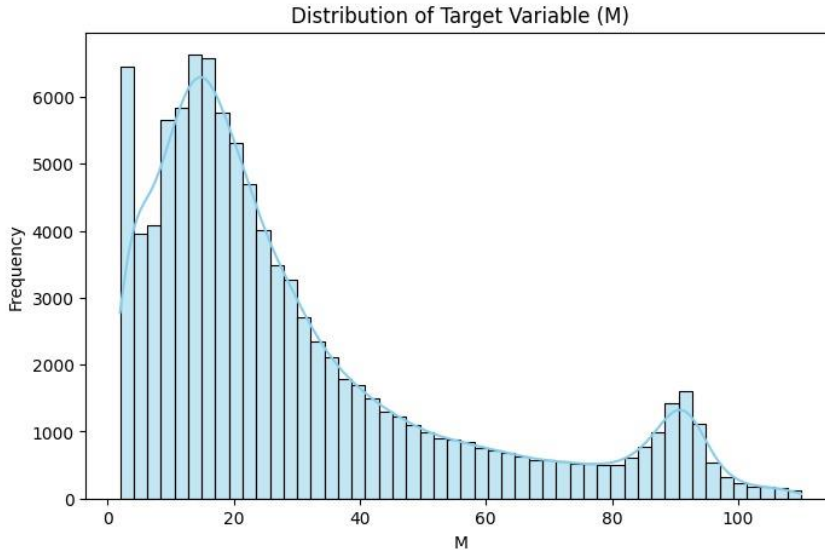


Figure 1.1: Distribution of Target variable (M)

This distribution has 2 peaks. One is between 10-20 GeV. This peak arises from,

$$e^+e^- \rightarrow \gamma^* \rightarrow l^-l^+$$

The intermediate particle is a virtual photon. This produces lepton pairs with lower invariant masses.

The peak around 90 GeV represents the Z boson resonance. (The mass of the boson is 91.2 GeV)

$$e^+e^- \rightarrow Z \rightarrow l^-l^+$$

In this analysis, the task was formulated as a regression problem. Linear regression, Random Forest regression (RF), Support Vector Regression (SVR), or other regression techniques can be employed to analyze this dataset. Linear regression performs well with linear relationships but is weak when non-linear relationships are present. SVR can handle both linear and non-linear relationships; however, its training speed is slow for large datasets. Random Forest regression, being an ensemble of multiple decision trees, can handle non-linear data while maintaining moderate training speed. Therefore, RF was selected as the primary machine learning technique for this study.

Additionally, a Deep Neural Network (DNN) was applied in this project. DNNs can capture complex, non-linear relationships, are robust to correlated inputs, and can handle large datasets. Since this dataset contains many correlated features, its high dimensionality can lead to overfitting. Principal Component Analysis (PCA) was applied to reduce the number of input features, remove redundancy, and simplify the models. Consequently, PCA was used to study the effect of dimensionality reduction on the performance of RF and DNN models.

Methods

Preprocessing

The original dataset consisted of 100,000 entries with 19 features. The following preprocessing steps were applied for Random Forest regression.

- **Remove ‘Event’ and ‘Run’ attributes**

The values of these attributes are not related to the target variable M . The ‘Run’ attribute could be used to split the dataset into subsets; however, the resulting subset sizes varied significantly. Therefore, the ‘Run’ attribute was removed, and the subsets were created based on size.

- **Check missing values and remove the rows with missing values.**

There were 85 rows with missing M values. Since the dataset is sufficiently large compared to this number, these rows were removed.

- **Check for duplicate and remove duplicate entries**

There were 23 duplicate rows and removed them.

- **Check for outliers using percentiles.**

1998 ~ 2% of the data found out of the 1st-99th percentile bounds. This was expected and there was no need to remove them.

- **Create composite attributes**

- Transverse Momentum Magnitude (pt_magnitude1 and pt_magnitude2)

$$pt_magnitude = \sqrt{px^2 + py^2}$$

- Absolute difference in pseudo rapidity (abs_eta_diff)

$$abs_eta_diff = |eta1 - eta2|$$

- Energy difference (energy_diff)

$$energy_diff = |E1 - E2|$$

- **Compute correlation matrix and remove least related attributes with M**

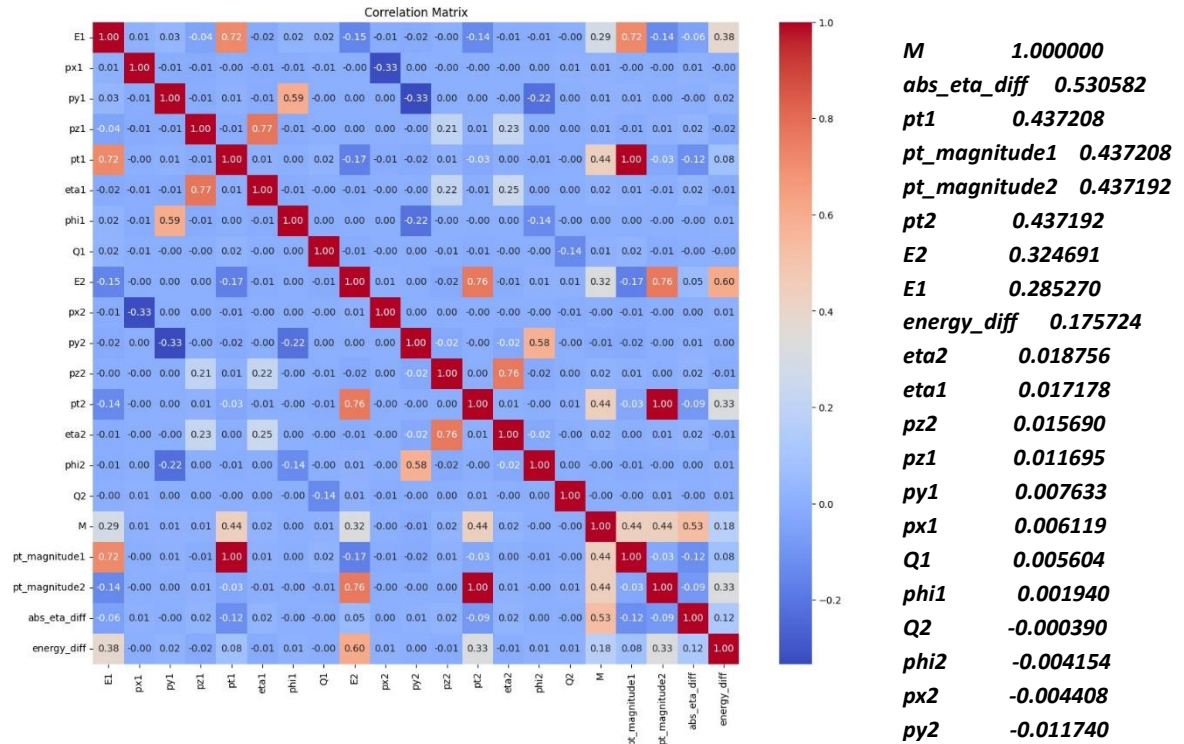


Figure 2.1: Correlation matrix and correlation with Target variable (M)

The attribute ‘abs_eta_diff’ exhibited the strongest correlation with M , while several features (px1, py1, phi1, Q1, px2, py2, phi2, and Q2) showed correlations lower than 0.01. Since their influence on M was minimal, these features were removed.

Feature scaling was not performed during this preprocessing step, as Random Forest regression does not require scaling.

After these preprocessing steps, the dataset consisted of 99,892 entries with 13 attributes, including M . The target variable and input features were then separated.

For training the Random Forest model, the dataset was split into training and test sets using an 80–20% split. The training set contained 79,913 entries, while the test set contained 19,979 entries. Since the training set was large, it was randomly shuffled and divided into five subsets, each

containing approximately 15,982 entries. Each model was then evaluated using the same test set as previously defined.

For the DNN, all subsets and the test set were scaled using a Standard Scaler prior to training.

Both types of models were evaluated using MAE, MSE, RMSE, and R^2 scores on both the training and test sets. Since the same test set was used for all ten models (five RF models and five DNN models), their performances could be directly compared.

Exploratory Data Analysis

The correlation matrix and the distribution of the target variable (M) were presented above. In addition, the distributions of the composite attributes were plotted. Since these four attributes are among those most strongly correlated with M , scatter plots showing their correlations with M were also generated

Random Forest Regression

Five separate Random Forest models were trained on the five non-scale subsets and evaluated using the same non-scaled test set. During hyperparameter tuning, the parameters ‘n_estimators’, ‘max_depth’, ‘min_samples_split’, and ‘min_samples_leaf’ were optimized. Three-fold cross-validation was employed for all models. Model performance was assessed using MAE, MSE, RMSE, and R^2 scores on both the training and test sets.

Deep Neural Network

Similar to the Random Forest models, five separate DNN models were trained and evaluated using the same test set. However, both the training and test sets were scaled prior to training. Each DNN model consisted of four layers with 64, 64, 32, and 1 units. The ReLU activation function was applied to the first three layers, while a linear activation function was used for the output layer. The ‘adam’ optimizer was employed due to its efficiency with large datasets. Mean squared error (MSE) was used as the loss function and mean absolute error (MAE) was used as an evaluation metric.

Early stopping was implemented to prevent overfitting, avoiding unnecessary epochs and ensuring that the model maintained its best performance. The number of epochs was set to 200, and a batch size of 64 was used. Model performance was evaluated using MAE, MSE, RMSE, and R^2 scores on both the training and test sets.

.

Principle Component Analysis

PCA was applied to the scaled training and test sets. Then training set was split into five subsets. In PCA, the parameter `n_components=0.95` was set to retain 95% of the variance. As a result, the 12 original features were transformed into 7 principal components. Subsequently, both Random Forest and DNN models were trained on each subset separately and evaluated using the same test set.

Results and Discussion

Exploratory Data Analysis

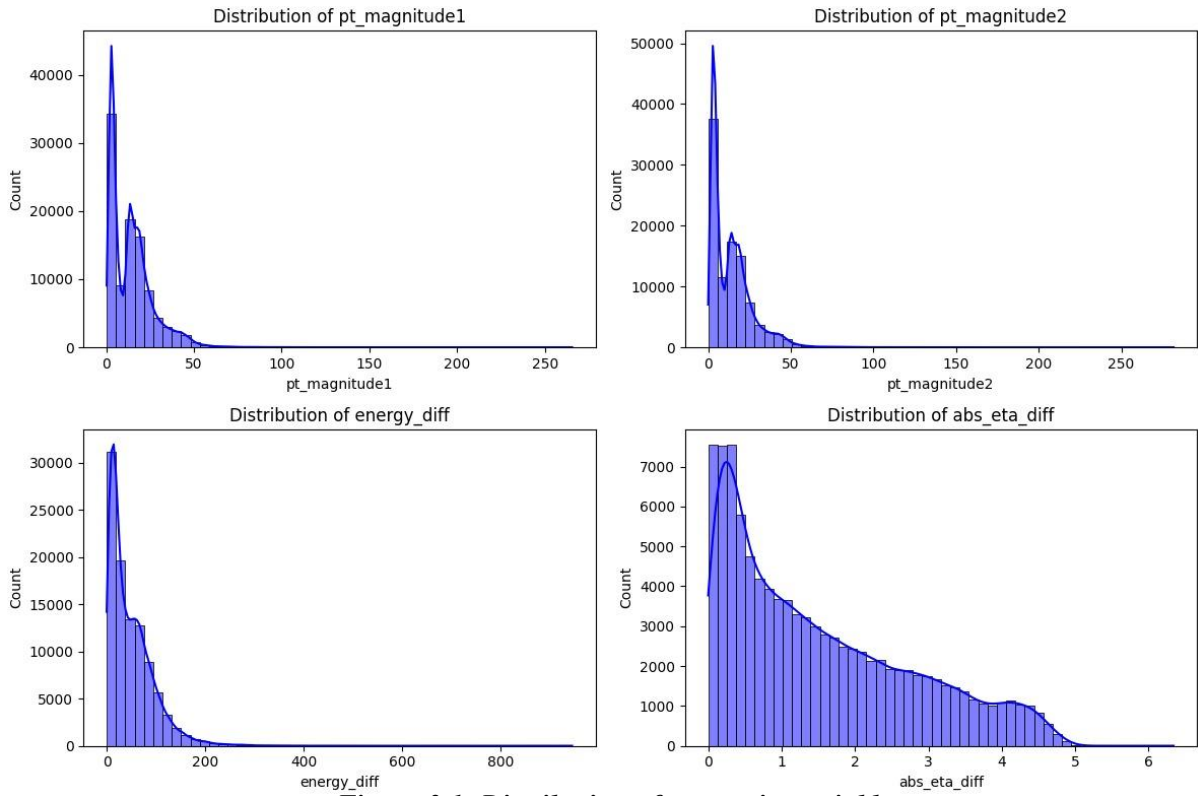


Figure 3.1: Distribution of composite variables

The transverse momentum ($pt_magnitude$) distributions exhibit a strong peak at low p_T and a long tail extending toward higher values, indicating that most collision events were produced by particles with low transverse momentum. The energy difference distribution also shows a sharp peak near zero and a long tail extending to larger values, suggesting that many events involve particles with nearly equal energies, which is typical of two-body decays. This observation supports the presence of Z-boson-like events in the dataset.

The distribution of the difference in pseudorapidity exhibits a peak at small values, gradually decreasing toward larger values. Small differences indicate that the particles are produced in similar angular directions.

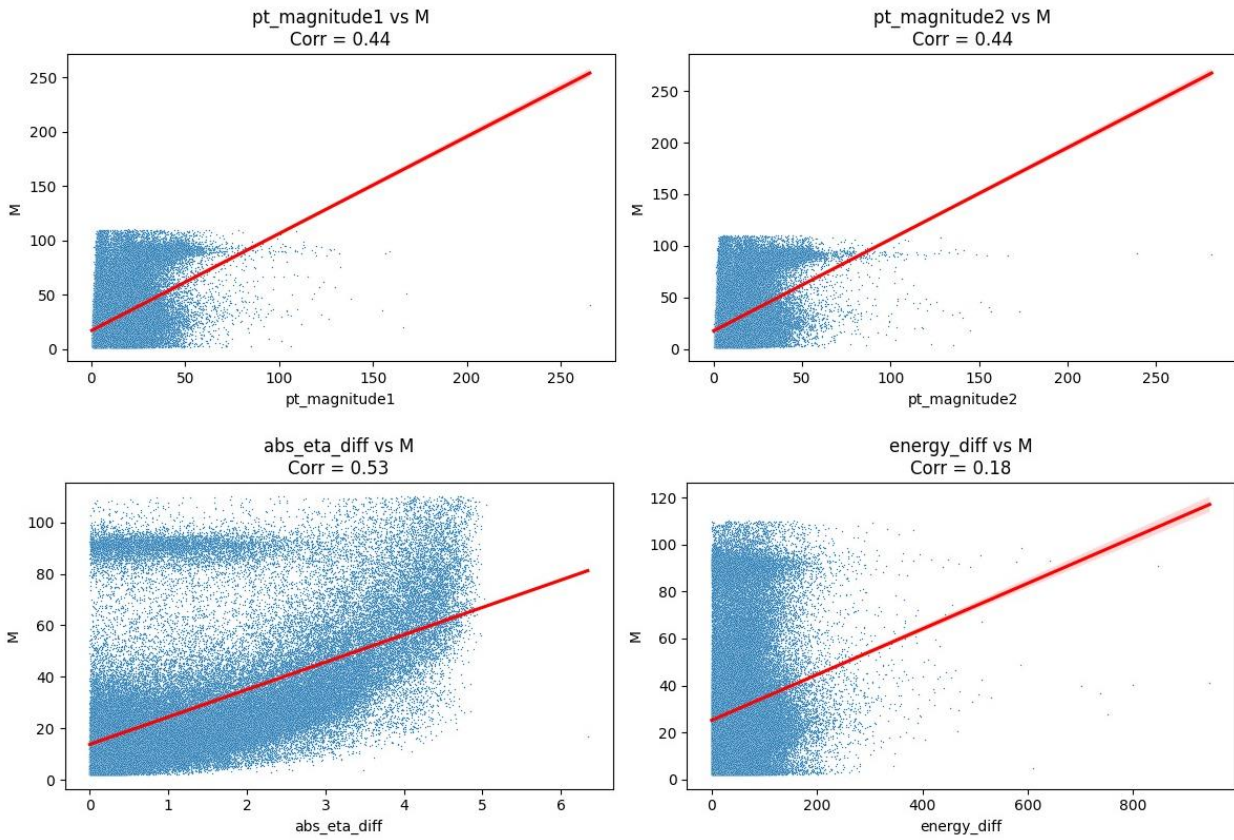


Figure 3.2: composite features vs M

These four scatter plots illustrate the relationship between the composite features and M . Both transverse momentum ($pt_magnitude$) features exhibit a moderate positive correlation with M , while abs_eta_diff shows the strongest correlation among them. This indicates that events with higher invariant mass are produced by particles with larger angular separation. The energy difference feature exhibits a weak positive correlation with M .

Random Forest Regression

The evaluation results of all five Random Forest models, tested using the same test set, are presented below.

metrics	Model 1		Model 2		Model 3		Model 4		Model 5	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
MAE	2.451	3.489	2.444	3.545	2.400	3.529	2.427	3.491	2.402	3.430
RMSE	3.576	5.082	3.571	5.117	3.505	5.162	3.558	5.121	3.571	5.031
R^2	0.980	0.959	0.980	0.959	0.981	0.958	0.980	0.959	0.979	0.960

The five Random Forest models exhibited consistent performance on both the training and test sets. The training set achieved an average R^2 score of 0.980, while the test set achieved an average R^2 score of 0.959, resulting in a difference of 0.021. This indicates slight overfitting, which is considered acceptable. The consistency of the results suggests that model performance is not sensitive to random initialization. Therefore, the Random Forest models successfully captured the non-linear correlations between the variables. However, the training process required significantly more time compared to the training of DNN models and Random Forest models with PCA.

Deep Neural Network

The evaluation results of all five DNN models, tested using the same test set, are presented below.

metrics	Model 1		Model 2		Model 3		Model 4		Model 5	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
MAE	3.201	3.555	3.348	3.522	3.308	3.543	3.275	3.550	3.367	3.455
RMSE	4.628	5.116	4.751	5.073	4.709	5.064	4.581	5.100	4.800	4.955
R^2	0.966	0.959	0.966	0.960	0.966	0.960	0.967	0.959	0.962	0.961

The five DNN models exhibited highly consistent performance on both the training and test sets. Overall, the training set evaluation results were slightly lower than those of the Random Forest models, while the test set evaluation results were comparable. However, the difference between the training and test set errors was much smaller than in the Random Forest models. The average R^2 score on the training set was 0.966, and on the test set it was 0.960, resulting in a difference of only 0.006. This indicates that the DNN models achieved a higher level of generalization than the Random Forest models and were almost perfectly fitted, without evidence of overfitting or underfitting.

Principle Component Analysis

After PCA was applied, the R^2 scores of both the Random Forest and DNN models changed as follows.

R^2	Model 1		Model 2		Model 3		Model 4		Model 5	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
RF	0.952	0.929	0.948	0.931	0.954	0.932	0.953	0.930	0.954	0.930
DNN	0.962	0.959	0.961	0.961	0.966	0.960	0.963	0.960	0.968	0.960

The Random Forest models exhibited less consistent performance after PCA was applied. The difference between the training and test set performances remained the same, indicating that generalization was not affected. However, the overall performance decreased, suggesting that PCA removed feature-specific information that the Random Forest models had been exploiting. The only advantage was a reduction in training time compared to the models trained on the original

feature space. Therefore, it can be concluded that Random Forest performs best on the original features.

The DNN models also showed slightly reduced consistency with PCA, but the effect was not as pronounced as for the Random Forest models. The average R^2 score on the training set was 0.964, and on the test set it was 0.960, resulting in a difference of 0.004, indicating better generalization than without PCA. This suggests that PCA did not remove important predictive information for the DNN models. Additionally, training was faster, model complexity was reduced, and generalization improved, representing an ideal outcome.

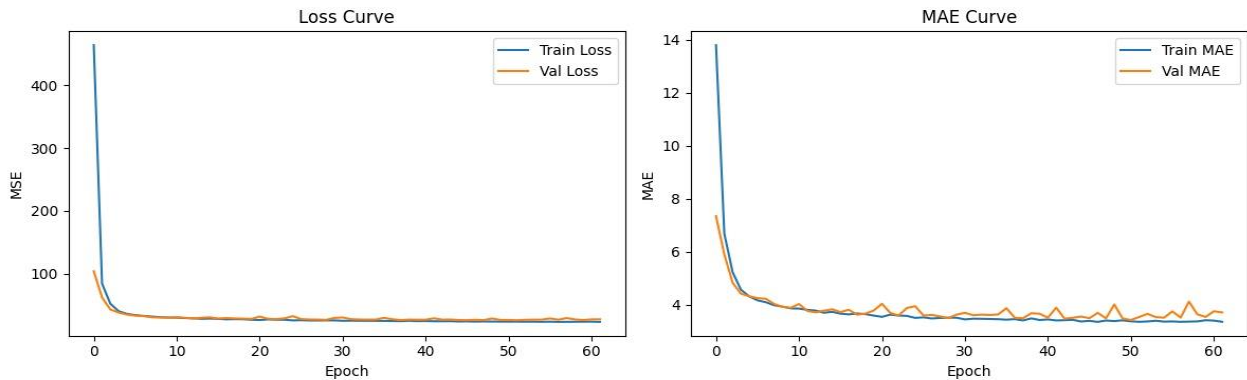


Figure 3.3: Training curves of DNN model 5 before PCA

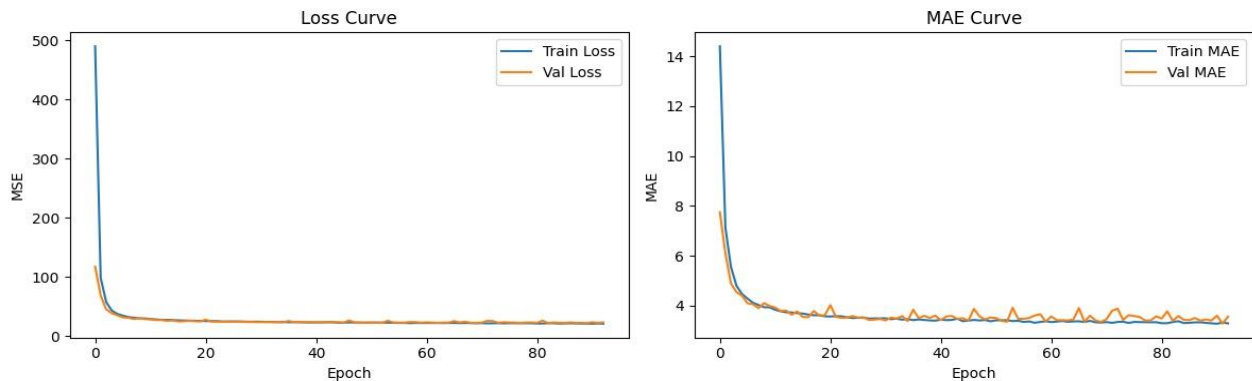


Figure 3.4: Training curves of DNN model 5 after PCA

In the loss curves, the training loss dropped sharply at the beginning, indicating that the network learned quickly. The validation loss also decreased, closely following the training loss. After PCA was applied, the fluctuations in the validation loss became even smaller, appearing almost flat and stabilized, which indicates the absence of overfitting or underfitting. This suggests that the model learned meaningful patterns.

Similarly, the MAE plots started at high values and rapidly decreased. The validation MAE followed a trajectory similar to that of the training MAE. After PCA was applied, the fluctuations in the validation MAE were smaller, indicating stable convergence. Overall, these plots demonstrate that the models were better fitted after PCA was applied.

Conclusion

In this study, the performances of Random Forest (RF) and Deep Neural Network (DNN) models, both with and without Principal Component Analysis (PCA), were compared for invariant mass prediction. The Random Forest models achieved very high training accuracy, reflecting their strong capability to capture complex non-linear relationships; however, some overfitting was observed, and substantially higher computational time was required. In contrast, the DNN models exhibited slightly lower training performance but achieved comparable test accuracy with much smaller train–test gaps, indicating superior generalization. After PCA was applied, the performance of the Random Forest models declined, suggesting that important feature-specific information had been removed. Conversely, the DNN models retained predictive accuracy while benefiting from reduced dimensionality, faster training, and improved generalization. Overall, the DNN models with PCA provided the most balanced and efficient modeling approach.